Materia: Programación III

UNIDAD 1 A 6 · DEL OBJETO A LA TRANSACCIÓN

PARCIAL

Parcial: la librería con acceso, transacciones y despliegue en el VPS

Objetivos

El parcial continúa el TP de la librería (capítulos 1 a 4) y lo lleva hasta el capítulo 6: al catálogo, los clientes y las ventas ya construidos se les agrega el acceso con usuarios y roles (capítulo 5), una capa de repositorios con Unit of Work que agrupa las escrituras de cada operación (capítulo 6) y, aparte, el despliegue del TP de los capítulos 1 a 4 en el VPS, dentro de un contenedor.

Al finalizar, el proyecto entregado debe demostrar que el grupo puede:

1. Reorganizar una API asincrónica en capas —**Router** → **Service** → **UoW** → **Repository** → **Model** — donde cada capa sabe lo suyo y nada más.
2. Evolucionar un esquema ya versionado con Alembic: una migración autogenerada y leída antes de aplicarla, y otra escrita a mano para lo que la comparación automática no ve.
3. Autenticar con contraseñas guardadas con bcrypt y un token en una cookie `HttpOnly`, sin frenar el servidor y sin revelar qué cuentas existen.
4. Autorizar consultando los roles **en la base** en cada petición, con vigencia, y distinguir siempre entre 401 y 403.
5. Agrupar en una sola transacción las escrituras de una operación de negocio, de modo que un fallo en el medio no deje datos a medias.
6. Delegar en la base las garantías que la base puede dar: unicidad entre los vigentes, claves foráneas, comprobaciones y el sólo agregado.
7. Empaquetar el TP de la librería (capítulos 1 a 4) en una imagen de Docker y ponerlo en línea en el VPS con una configuración que no filtra secretos ni expone lo que no debe.

> **CÓMO SE ENTREGA**
>
> La entrega es **una por grupo**, en formato `.zip`, y la sube un solo integrante al campus. El `.zip` va **sin las carpetas** `.venv` **ni** `__pycache__`, y tampoco lleva el archivo `.env`. El detalle de lo que contiene está en el formato de entrega, al final.

QUÉ QUEDA AFUERA

No se evalúa la concurrencia (capítulo 7). Dos ventas del último ejemplar de un libro, registradas al mismo tiempo, pueden leer las dos `stock = 1`, darlo las dos por suficiente y escribir las dos `0`: el segundo descuento pisa al primero y la librería vende un ejemplar que no tenía, sin que el stock llegue a ser negativo —el CHECK lo impediría—. Es una **actualización perdida**: el nivel de aislamiento por defecto la permite, y el bloqueo que la evita es tema de la próxima clase. Tampoco hay colas de tareas con Redis (capítulo 8): el aviso al cliente se resuelve con `BackgroundTasks`, que corre en el mismo proceso. Si el proceso se reinicia, el aviso se pierde y nadie lo reintenta. Acá se acepta porque el aviso es simulado; el capítulo 8 lo resuelve.

Qué cambia respecto del TP de los capítulos 1 a 4

El grupo **parte de su propio TP entregado**. Esta tabla resume lo que se agrega y lo que se modifica; el detalle está en las secciones siguientes.

Qué	Cambio
Usuarios, roles e intentos de acceso	**Nuevo.** Tablas `usuario`, `asignacion_rol`, `intento_acceso` y `evento_seguridad`; módulo `usuarios`
`cliente`	Se agrega `usuario_id`, nulo permitido y UNIQUE: un cliente puede tener, o no, acceso por la web
`libro`	Se agrega el borrado lógico (`borrado_en`). El ISBN deja de ser un UNIQUE común y pasa a ser único **entre los vigentes**
Stock	**Nuevo** `movimiento_stock`, de sólo agregado. El stock ya no se edita por PATCH: cambia con el alta, la reposición y las ventas
Registro de una venta	Además de crear la venta y sus renglones, **descuenta el stock** de cada libro y deja un movimiento por renglón, todo en una sola transacción
Capas	Todos los módulos pasan a **repositorios + Unit of Work**; los servicios dejan de recibir la sesión
Configuración	`Settings` con pydantic-settings, sin secretos por defecto
Listados	Devuelven el envoltorio `{items, total, page, size, pages}` en lugar de `limit y offset`
Despliegue	**Nuevo, y aparte.** El TP de los capítulos 1 a 4, tal como se entregó, se empaqueta en una imagen con un `Dockerfile`, se corre con `docker run` y se pone en línea en el VPS. No lleva ninguna de las demás modificaciones de esta tabla

Contexto

La librería ya tiene su catálogo, sus clientes y sus ventas. Ahora abre el sistema a su personal y a sus clientes. Un **administrador** mantiene el catálogo, repone el stock y administra al personal y sus roles. Un **vendedor** registra las ventas y atiende a los clientes en el mostrador. Los **clientes** se registran solos en la web y consultan sus compras. El catálogo de libros es **público**: se puede ver sin iniciar sesión.

El dominio está elegido para que cada capítulo tenga un lugar concreto donde aparecer:

Lo que pide el dominio	Por qué	Capítulo
Tres roles, que el administrador asigna y revoca con vigencia	La autorización se decide en la base, no en el token	5
El registro crea el usuario, su rol y su ficha de cliente	Tres escrituras que no tienen sentido por separado	6
Registrar una venta descuenta el stock de cada libro	Cuatro clases de escritura en una sola transacción: venta, renglones, stock y movimientos	6
Un libro descatalogado se da de baja, pero sus ventas viejas siguen mostrándolo, y el ISBN se puede volver a usar	Borrado lógico e índice único parcial	4 y 6
El límite de intentos de login se calcula contando una tabla	El intento tiene que sobrevivir al rollback del login fallido	5 y 6
Los movimientos de stock no se corrigen: se compensan con otro	Sólo agregado, impuesto por la base	6
El TP de la librería tiene que estar al alcance de quien lo use	Se despliega en un servidor, dentro de un contenedor	1 a 4

Requisitos técnicos

Componente	Versión mínima	Uso
Python	3.12	Lenguaje
FastAPI	0.111	Framework web; todos los handlers con `async def`
SQLModel	0.0.24	Modelos de tabla
SQLAlchemy	2.0, asincrónico	Motor, sesión y carga anticipada
asyncpg	0.30	Controlador asincrónico de PostgreSQL

Componente	Versión mínima	Uso
PostgreSQL	16	Base de datos; en el despliegue, la que ya tiene el servidor
Alembic	1.13, plantilla asincrónica	Migraciones
pydantic-settings	2.0	Configuración leída del entorno
bcrypt	4.0	Hash de las contraseñas
PyJWT	2.8	Token firmado
uvicorn	El que instala `fastapi[standard]`	Servidor ASGI
Docker	20.10	Construir la imagen y correr el contenedor del TP de los capítulos 1 a 4, en la máquina del grupo y en el VPS

La configuración se lee con una clase `Settings` de pydantic-settings, desde las variables de entorno o un archivo `.env` que no se versiona. La dirección de la base se arma con las variables sueltas `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `POSTGRES_HOST` y `POSTGRES_PORT`, con el driver `postgresql+asyncpg`. La clave de firma de los tokens también sale del entorno, y ni ella ni la contraseña de la base tienen un valor por defecto en el código: si faltan, la aplicación no arranca. La razón es que la clave de firma es el secreto más importante del sistema: quien la tenga fabrica un token válido para cualquier usuario (sección 5.12). Y una clave inventada al arrancar tampoco sirve: cambia en cada reinicio, y con dos procesos atendiendo cada uno rechaza los tokens del otro. El repositorio incluye un `env.example` con todas las variables y ningún secreto.

Modelo de dominio

Los nombres de tablas y columnas son obligatorios: los casos de prueba y la corrección los usan. Se pueden agregar columnas, pero no quitar ni renombrar las indicadas. Las tablas que no aparecen en esta sección (`editorial`, `genero`, `autor`, `libro_autor` y `perfil_cliente`) **no cambian** respecto del TP.

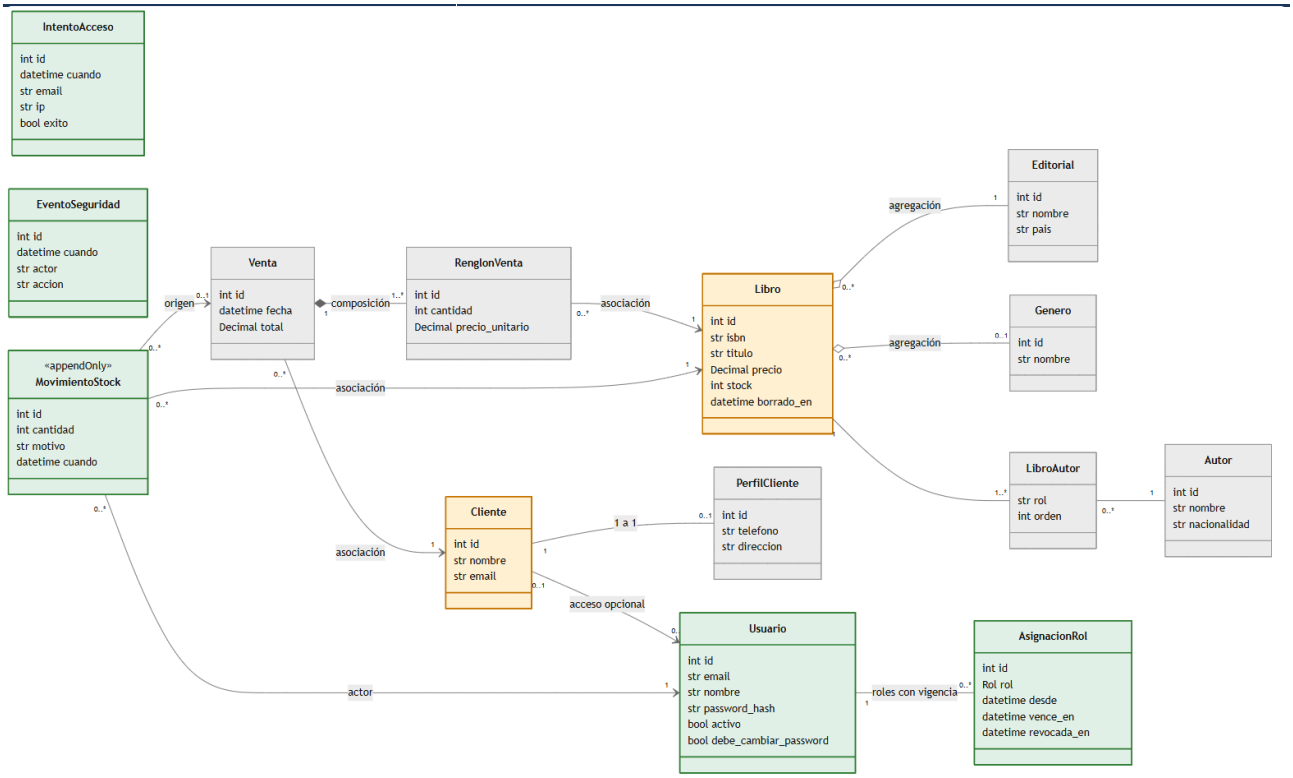


Figura 1 · Modelo completo: el del TP de los capítulos 1 a 4 más lo que agrega el parcial. En verde, las clases nuevas; en ámbar, las que cambian (`Libro` incorpora el borrado lógico y `Cliente` , el acceso opcional); en gris, las que no cambian. El rombo lleno es composición, el rombo vacío es agregación y la flecha es asociación. «appendOnly» marca la tabla de sólo agregado que protege el disparador. `IntentoAcceso` y `EventoSeguridad` no se relacionan con ninguna otra tabla.

Tabla	Columnas	Restricciones
`usuario`	`id` , `email` , `nombre` , `password_hash` , `activo` , `debe_cambiar_password`	`email` UNIQUE
`asignacion_rol`	`id` , `usuario_id` , `rol` , `desde` , `vence_en` (nulo), `revocada_en` (nulo), `otorgada_por` (nulo)	CHECK de `rol` dentro de ADMIN, VENDEDOR, CLIENTE; **índice único parcial** (`usuario_id` , `rol`) `WHERE revocada_en IS NULL`
`intento_acceso`	`id` , `cuando` , `email` , `ip` , `exito`	Índice (`email` , `ip` , `cuando`)
`evento_seguridad`	`id` , `cuando` , `actor` (texto), `accion`	Sin restricciones extra
`cliente` (cambia)	Se agrega `usuario_id` (nulo permitido)	`usuario_id` UNIQUE; `email` sigue UNIQUE
`libro` (cambia)	Se agrega `borrado_en` (con zona, nulo)	**Índice único parcial** (`isbn`) `WHERE borrado_en IS NULL` , en lugar del UNIQUE; siguen los CHECK `precio > 0` y `stock ⩾ 0`

Tabla	Columnas	Restricciones
`movimiento_stock`	`id`, `libro_id`, `venta_id` (nulo), `usuario_id`, `cantidad`, `motivo`, `cuando`	CHECK `cantidad <> 0`; **sólo agregado**

Todas las fechas nuevas son `timestamptz`. El dinero sigue siendo `NUMERIC` y en Python `Decimal`, nunca `float` (sección 2.7). Cada clave foránea nueva lleva un índice, porque PostgreSQL no lo crea solo (sección 4.10). La navegación se declara en los dos extremos, como en el TP.

EL ISBN DEJA DE SER UN UNIQUE COMÚN

`libro` ahora tiene borrado lógico: la fila dada de baja sigue en la tabla, porque las ventas viejas la apuntan. Con un UNIQUE común, la librería no podría volver a cargar un libro con un ISBN que dio de baja, y la base respondería «ya existe» de un libro que la aplicación no muestra en ningún listado. Por eso la unicidad es sólo entre los vigentes: un índice único parcial con `WHERE borrado_en IS NULL`, la misma herramienta que usa `asignacion_rol` (sección 4.9).

LOS ROLES VIVEN EN SU PROPIA TABLA

`usuario` no tiene una columna `rol`. Los roles de una persona son las filas de `asignacion_rol` que están **vigentes**: no revocadas y sin vencer. Revocar un rol escribe `revocada_en`, y la fila no se borra nunca: así queda el historial de quién tuvo qué rol y hasta cuándo (sección 5.9).

EL STOCK TIENE UNA SOLA VÍA DE ESCRITURA

`libro.stock` es el resultado de sumar los movimientos de ese libro. Por eso siempre debe cumplirse que, para cada libro, `stock` **es igual a la suma de la** `cantidad` **de sus movimientos**. Esa igualdad es la que se verifica después de cada caso de venta, y se rompe si alguna parte del código escribe el stock sin dejar un movimiento.

Reglas de negocio

Las reglas de esta tabla se **numeran de nuevo** y reemplazan a las del TP en lo que se superponen. Siguen vigentes, aunque no se repitan acá, los CHECK de precio y de stock del libro y la búsqueda por título con parámetros ligados.

Regla	Enunciado	Garante esperado
RN-01	El correo de un usuario, el de un cliente, el nombre de una editorial y el de un género son únicos. El ISBN de un libro es único entre los que no están dados de baja.	UNIQUE e índice único parcial en la base; 409 con mensaje propio
RN-02	El auto-registro crea el usuario, su rol CLIENTE y su ficha de cliente: o quedan las tres filas, o ninguna. Si ya existe un cliente con ese correo, el registro da 409 y no se apropia de esa ficha.	Un solo Unit of Work
RN-03	Un usuario tiene a lo sumo una asignación abierta por rol.	Índice único parcial; 409
RN-04	Los permisos se deciden con los roles vigentes en la base, en cada petición. Los roles que trae el token son informativos.	Dependencia que consulta `asignacion_rol`
RN-05	Un administrador no puede revocarse su propio rol ADMIN, y el sistema nunca queda sin administradores vigentes.	Comprobación en el servicio; 409
RN-06	Mientras un usuario tenga el cambio de contraseña pendiente, sólo puede usar las rutas de `/auth` y las que no piden sesión.	Dos grupos de rutas; 403
RN-07	Después de 5 intentos fallidos con el mismo correo desde la misma dirección en 60 segundos, el login responde 429 sin verificar la contraseña. Un login correcto reinicia la cuenta: los fallos anteriores siguen en la tabla, pero dejan de contar.	Conteo sobre `intento_acceso`; `Retry-After`
RN-08	El login responde lo mismo, y tarda lo mismo, exista o no el correo.	Mismo mensaje; verificación contra un hash ficticio
RN-09	Una venta nace con sus renglones (al menos uno, sin libros repetidos), con el precio de cada renglón copiado del libro y el total calculado por el servidor. Las ventas no se modifican.	DTO; el precio y el total nunca viajan en el cuerpo; no hay PATCH
RN-10	Registrar una venta descuenta el stock de cada libro y deja un movimiento por renglón. Si un libro no alcanza, **no queda ninguna** de esas escrituras: ni la venta, ni los renglones, ni los descuentos, ni los movimientos.	Un solo Unit of Work; 409
RN-11	El stock de un libro sólo cambia a través de movimientos: el alta con stock inicial, la reposición y la venta. No se edita por PATCH.	Punto único de escritura de stock
RN-12	`movimiento_stock` es de sólo agregado: nadie puede modificar ni borrar sus filas.	Disparador en la base, instalado por migración
RN-13	Un libro dado de baja deja de aparecer en el catálogo y no se puede vender ni reponer, pero sus datos se conservan y las ventas anteriores lo siguen mostrando.	Borrado lógico; 409

Regla	Enunciado	Garante esperado
RN-14	Lo propio sólo pide estar autenticado: un cliente ve sus ventas y ninguna otra.	`/mis-ventas` y comprobación de dueño; 403
RN-15	Ningún error devuelve al cliente texto interno de la base, que le entregaría el mapa del esquema (sección 4.13). Ninguna entrada inválida termina en un 500.	Traducción del `IntegrityError`; validación en el contrato
RN-16	Cada alta de personal, cambio de rol y cambio de contraseña deja una línea en `evento_seguridad`, en la misma transacción que el cambio.	Un solo Unit of Work

Estructura obligatoria del proyecto

La organización es por módulo funcional, con las mismas capas en todos los módulos:

Ruta	Contenido
`main.py`	Aplicación, lifespan y los dos grupos de rutas
`seed.py`	Deja la base en el estado inicial conocido
`app/core/config.py`	`Settings` con pydantic-settings
`app/core/database.py`	Las funciones que crean el motor y la fábrica de sesiones (las llama el lifespan), y `get_session`
`app/core/security.py`	Hash con bcrypt fuera del bucle, y emisión y verificación del token
`app/core/dependencies.py`	Usuario actual, roles vigentes, control del cambio obligatorio y `requiere_rol`
`app/core/repository.py`	`BaseRepository[T]` y `SoftDeleteRepository[T]`
`app/core/paginacion.py`	El envoltorio de los listados y el cálculo de las páginas
`app/core/unit_of_work.py`	`UnitOfWork` base: confirma o revierte al salir del bloque
`app/modules/<módulo>/models.py`	Modelos de tabla, restricciones y relaciones
`app/modules/<módulo>/schemas.py`	Contratos de entrada y de salida
`app/modules/<módulo>/repository.py`	Los repositorios de cada entidad del módulo
`app/modules/<módulo>/unit_of_work.py`	El Unit of Work del módulo, con los repositorios que sus operaciones necesitan

Ruta	Contenido
`app/modules/<módulo>/service.py`	Las reglas de negocio. Recibe el Unit of Work y lo abre
`app/modules/<módulo>/router.py`	Rutas HTTP: construyen el Unit of Work, llaman al servicio y traducen errores
`migrations/`	Configuración de Alembic y sus versiones
`test/`	Archivos `.http` y `solo_agregado.sql` con los casos de prueba

Módulos mínimos: `health`, `usuarios`, `editoriales`, `generos`, `autores`, `libros`, `clientes` y `ventas`. `AsignacionRol`, `IntentoAcceso` y `EventoSeguridad` viven en `usuarios`; `MovimientoStock` en `libros`; `LibroAutor` en `libros`, `PerfilCliente` en `clientes` y `RenglonVenta` en `ventas`. El módulo `health` sólo tiene `router.py`. El despliegue **no forma parte de esta estructura**: se hace sobre el TP de los capítulos 1 a 4, en una carpeta aparte (ver el formato de entrega).

Consignas

Ejercicio 1 · Proyecto, configuración y ciclo de vida

6 puntos

La configuración se lee con `Settings` desde el entorno y la dirección de la base se arma con un `@computed_field`. El motor asincrónico, con el tamaño del pool declarado en la configuración (sección 4.5), y la fábrica de sesiones se crean en el lifespan y se guardan en `app.state`, no al importar un módulo (sección 1.9): un recurso construido al importar se crea antes de que exista el bucle de eventos, y el síntoma —anda al levantar el servidor y falla en las pruebas, o al revés— no señala la causa. `get_session` toma la fábrica de `app.state`, entrega una sesión por petición, creada con `expire_on_commit=False` (sección 4.5), y la cierra al terminar. El lifespan calcula además, una vez, el hash ficticio del login, y al apagar cierra el motor con `dispose()`. Se exponen `GET /health/live`, que no toca la base, y `GET /health/ready`, que ejecuta `SELECT 1` y responde 503 si la base no contesta: el proceso puede estar vivo y no estar listo, y reiniciarlo no arregla una base caída.

Ejercicio 2 · Modelos, relaciones y migraciones

10 puntos

Escribir los modelos nuevos y modificar `Cliente` y `Libro` según el modelo de dominio, con todas sus restricciones, incluidos los dos índices únicos parciales (`postgresql_where`): el de `asignacion_rol` y el del ISBN. `Cliente.usuario` devuelve un objeto y no una lista. El esquema se crea **exclusivamente** con Alembic: no se usa `create_all`. A las migraciones del TP se suman **dos nuevas**. La tercera se genera con `--autogenerate` y trae las tablas de acceso, el movimiento de stock y los cambios de `cliente` y `libro`. La cuarta se **escribe a mano** y trae el disparador de sólo agregado sobre `movimiento_stock` (RN-12), porque la comparación automática no ve funciones ni disparadores. Cada migración nueva se lee antes de aplicarla y lleva un comentario que lo diga; si la versión generada tiene un problema, se corrige a mano y el comentario lo explica. `alembic upgrade head` sobre una base vacía, `alembic downgrade base` y otra vez `upgrade head` deben funcionar.

Ejercicio 3 · Contratos y CRUD

8 puntos

Implementar los endpoints de referencia con contratos separados de los modelos de tabla. Los contratos de entrada rechazan los campos que no declaran (`extra="forbid"`), de modo que nadie se registra con un rol ni edita el stock por PATCH (RN-11). Ninguna respuesta incluye `password_hash`. Los PATCH aplican sólo los campos enviados (`exclude_unset`, sección 2.6). Los códigos son 201 al crear, 204 al dar de baja un libro, 404 si el recurso de la ruta no existe, 409 si choca con lo guardado y 422 si el cuerpo o los parámetros no cumplen el contrato (sección 2.8). Los listados devuelven el envoltorio de la sección 2.9, `{items, total, page, size, pages}`: `page` empieza en 1, `size` va de 1 a 100 (por defecto 20), pedir una página más allá de la última da 422, y `total` cuenta con **los mismos filtros** que la página. Con cero resultados hay una página, vacía. La búsqueda de libros por título sigue usando parámetros ligados y sin distinguir mayúsculas.

Ejercicio 4 · Autenticación

14 puntos

Las contraseñas se guardan con bcrypt, con un costo declarado en la configuración y no fijo en el código, porque es un valor que se sube con los años (secciones 5.4 y 5.12), y toda llamada a bcrypt corre fuera del bucle de eventos — `to_thread` — y bajo un semáforo que limita cuántas corren a la vez (sección 5.6). El hash se calcula **antes** de abrir la transacción, porque adentro la conexión quedaría tomada los 300 ms que tarda bcrypt sin usar la base. Todo campo que termina en bcrypt —también la contraseña del login y la actual del cambio— acepta como máximo 72 bytes: bcrypt no mira más allá, y desde la versión 5 rechaza con una excepción lo que pasa de ahí, así que sin el límite en el contrato una contraseña larga daría un 500 en lugar de un 422. Son bytes y no caracteres: una «ñ» ocupa dos. El login sigue este orden (sección 5.5): primero el límite de intentos (RN-07), que cuesta una consulta y, si se superó, responde 429 con `Retry-After` sin calcular ningún hash; después la verificación, contra el hash del usuario o contra un hash ficticio si el correo no existe (RN-08). El token es un JWT firmado con `sub`, `exp` y los roles vigentes al emitirlo (sección 5.3). Viaja **sólo** en una cookie `access_token` con `HttpOnly`, `SameSite=Lax`, `Max-Age` igual a la duración del token y `Secure` configurable; el cuerpo del login no lo trae. La dependencia lo lee de `request.cookies`. Es un login tradicional con cookie: la sesión dura hasta que el token vence o se cierra la sesión, y cambiar la contraseña no invalida las sesiones abiertas ni emite un token nuevo. Existen el registro (RN-02), el cierre de sesión, que borra la cookie, y `GET /auth/yo`, que devuelve el usuario, sus roles vigentes y los que dice el token.

Ejercicio 5 · Autorización

12 puntos

`get_usuario_actual` responde 401 si no hay cookie, si el token es inválido o venció, o si el usuario no existe o está inactivo. `requiere_rol` responde 403 y consulta los roles **vigentes en la base** en cada petición (RN-04): revocar un rol le quita el permiso a una sesión ya abierta, sin esperar a que el token venza (sección 5.9). El administrador asigna roles con vencimiento opcional y los revoca sin borrar la fila; reasignar un rol vencido cierra la fila anterior y abre una nueva. El vencimiento llega con zona horaria y en el futuro: una fecha sin zona es ambigua —¿la hora de quién?— y una que ya pasó no choca con nada guardado; las dos son 422, no 409. Se cumple la autoprotección (RN-05). El cambio obligatorio de contraseña se resuelve con **dos grupos de rutas** en `main.py` —uno abierto y uno protegido, que lleva el control montado una sola vez— porque FastAPI suma dependencias y no las resta (sección 5.10). Lo propio sólo pide estar autenticado (RN-14). El historial de seguridad (RN-16) registra los datos iniciales con el actor `SISTEMA`, que es un texto y no un usuario (sección 5.8).

Ejercicio 6 · Repositorios y Unit of Work

16 puntos

`BaseRepository[T]` recibe en su constructor la sesión y **la clase del modelo**, porque los genéricos se borran en tiempo de ejecución (sección 6.4.3), y ofrece al menos `get_by_id`, `get_all`, `count`, `add`, `update` y `delete`. `SoftDeleteRepository[T]` hereda de él, lo usa `Libro`, filtra los dados de baja y agrega `get_by_id_including_deleted` y `soft_delete`, que no existen en la clase base (sección 6.4.2). `add` y `update` hacen `flush()`, nunca `commit()`: **ningún repositorio confirma** (sección 6.4.4). Cada entidad tiene su repositorio, con métodos que se nombran por lo que traen, y **la precarga se declara adentro del método que la necesita** (sección 6.5). El `UnitOfWork` base es un gestor de contexto asincrónico: confirma al salir del bloque, revierte si se sale con una excepción y **no cierra la sesión**, porque no la creó (secciones 6.6 y 6.7). Cada módulo tiene su Unit of Work, que compone los repositorios que sus operaciones necesitan, todos con la misma sesión. El router construye el Unit of Work con una dependencia y se lo pasa al servicio sin abrirlo; el servicio lo abre con `async with` y nunca lo construye. Las operaciones de RN-02, RN-10 y RN-16 se escriben cada una en un solo bloque. Las dependencias del usuario actual y de verificación de rol reciben la **misma sesión** que el Unit of Work: sin esa unificación, cada petición autenticada tomaría dos conexiones del pool (sección 6.7). El stock se escribe en **un único punto** (RN-11), que usan el alta del libro, la reposición y el registro de ventas.

Ejercicio 7 · Integridad: sólo agregado e intento de acceso

10 puntos

El disparador de la cuarta migración rechaza cualquier `UPDATE` o `DELETE` sobre `movimiento_stock`, venga de donde venga (RN-12, sección 6.9). Cada intento de login, correcto o fallido, se anota en `intento_acceso` desde **una sesión y una transacción propias**, que se confirman siempre: es la única escritura del sistema que no comparte la transacción de su petición, porque el fallo que se quiere contar es el mismo que la revierte (sección 6.8.1). El código lo declara en un comentario que dice tres cosas: qué hace distinto, por qué no puede compartir la transacción y cuál es su costo —deja filas aunque la petición falle—. La sección 6.9 describe un segundo mecanismo independiente para el sólo agregado —que el rol de base con el que se conecta la aplicación no tenga permiso de modificar esa tabla—; no se pide, porque el parcial usa un solo rol de base, y según esa misma sección cualquiera de los dos alcanza: tener los dos es defensa en profundidad.

Ejercicio 8 · Lecturas sin N+1 y asincronía

6 puntos

Toda respuesta que incluya datos de una relación los obtiene con carga anticipada declarada en el repositorio. El detalle de una venta —con su cliente, los renglones y el libro de cada renglón, incluso si el libro está dado de baja— se resuelve con un número fijo de consultas, que no crece con la cantidad de renglones (sección 4.6). Ninguna lectura depende de la carga perezosa (sección 3.8). Todos los handlers son `async def` y ninguno ejecuta una llamada bloqueante (sección 3.6). Al registrar una venta se agenda con `BackgroundTasks` el aviso del comprobante al cliente, simulado con `await asyncio.sleep(0.5)` y un mensaje en el registro, que corre **después** de enviar la respuesta y sólo si la operación se confirmó (sección 3.9). El mensaje tiene que verse en la consola con el comando de arranque normal de uvicorn: con el registro por defecto, el de un logger propio sin configurar no aparece.

Ejercicio 9 · Contenedor: la imagen

11 puntos

Se empaqueta el **TP de los capítulos 1 a 4**, tal como se entregó —la librería con su catálogo, sus clientes y sus ventas, sin los cambios de este parcial— en una **imagen de Docker**, construida con un `Dockerfile` y puesta a correr con `docker run`. No se usa Docker Compose: la base de datos no forma parte del paquete, porque el servidor ya tiene PostgreSQL. Se le agregan al TP sólo el `Dockerfile` y el `.dockerignore`, y su lógica no se toca. Si el TP no arranca en el contenedor por un defecto propio (por ejemplo, una ruta de salud que no responde como pedía su consigna), se corrige y se anota en `DEPLOY.md`. Esta consigna es lo que se entrega en el repositorio; la siguiente, lo que tiene que estar en línea.

La imagen. El `Dockerfile` parte de `python:3.12-alpine`. La razón del límite de 100 MB es concreta: el VPS descarga y guarda esa imagen, cada megabyte de la base se paga en disco y en tiempo de descarga en cada despliegue, y cada paquete del sistema que la imagen trae es superficie de ataque que la aplicación nunca usa. Medida con Docker 29.2 el 2 de noviembre de 2026, `python:3.12-alpine` ocupa **74,7 MB** en disco; `python:3.12-slim`, la alternativa habitual, ocupa **180 MB** y por eso no cumple. El límite es de la **imagen base**: la del proyecto, con sus dependencias instaladas, es bastante más grande, y eso es lo esperado. Alpine usa musl en lugar de glibc, y eso suele traer problemas con paquetes que se compilan al instalarse; se verificó que las dependencias del stack del TP (asyncpg, pydantic-core y el resto de su `requirements.txt`) se instalan en esa imagen sin compilar nada, así que no hace falta instalar ningún compilador.

El `Dockerfile` cumple lo siguiente, y cada punto tiene su razón:

- Copia y instala `requirements.txt` **antes** de copiar el código, con `--no-cache-dir`: mientras las dependencias no cambien, esa capa sale de la caché y un cambio en el código no reinstala todo.
- La aplicación **no corre como root**: se crea un usuario sin privilegios y se lo activa con `USER`. Si alguien vulnera la aplicación, no es dueño del contenedor.
- **La imagen no contiene secretos.** El `.dockerignore` excluye `.env`, `.venv`, `.git` y los `__pycache__`, y ni la dirección de la base ni su contraseña aparecen en el `Dockerfile`: entran como variables de entorno al correr el contenedor. Lo que se hornea en una capa queda ahí para siempre, aunque una capa posterior lo borre.
- Al arrancar, el contenedor aplica las migraciones del TP con `alembic upgrade head` y después levanta uvicorn con `app.main:app` escuchando en `0.0.0.0` (dentro de un contenedor, `127.0.0.1` sólo se alcanza desde el propio contenedor, y el puerto publicado nunca respondería). Si la base no responde, Alembic falla y el contenedor termina con error: es lo que se quiere, porque una aplicación que no llega a su base no debería quedar atendiendo.
- Declara un `HEALTHCHECK` sobre `GET /health/ready`. Sin Compose, es la propia imagen la que dice cuándo está sana, y `docker ps` lo muestra.

Cómo se corre. La imagen se pone a andar con un comando de esta forma:

```
docker run -d --name libreria -p <puerto>:8000 --restart unless-stopped \
  --env-file <archivo-de-variables> libreria
```

- `-p <puerto>:8000` publica **sólo la aplicación**, en el puerto que asigna la cátedra.
- `--restart unless-stopped` hace que Docker vuelva a levantar el contenedor si termina: por ejemplo, si el servidor se reinicia, o si la base todavía no respondía cuando arrancó.
- `--env-file` entrega la configuración desde un archivo que vive **en el servidor**, fuera del repositorio y de la imagen. Tiene `DATABASE_URL` —con el driver `postgresql+asyncpg`, y el servidor, el usuario, la contraseña y el nombre de la base que provee la cátedra— y `SQL_ECHO=false`, que deja de escribir cada consulta en el registro.

> **LA BASE NO ESTÁ EN `LOCALHOST`**
>
> Dentro de un contenedor, `localhost` es el propio contenedor y no el servidor: una `DATABASE_URL` con `localhost` falla con «Connection refused» aunque PostgreSQL esté funcionando. Si la base corre directamente en el servidor, el contenedor la alcanza por el nombre `host.docker.internal`, que en Linux hay que habilitar con `--add-host=host.docker.internal:host-gateway` en el `docker run`. Si la cátedra entrega otra dirección, se usa esa.

> **EL HEALTHCHECK Y `LOCALHOST`**
>
> Dentro de una imagen Alpine no hay `curl`; `wget`, que sí viene, resuelve `localhost` primero a `::1` (IPv6), y uvicorn con `--host 0.0.0.0` escucha sólo en IPv4. Un `HEALTHCHECK` con `http://localhost:8000/...` falla con «Connection refused» aunque el servidor responda perfectamente desde afuera, y el contenedor queda `unhealthy` sin que nada parezca roto. Se comprueba con `docker exec libreria wget -qO- http://127.0.0.1:8000/health/live`, y la dirección que sirve es `127.0.0.1`.

Ejercicio 10 · Despliegue en el VPS

7 puntos

El TP de los capítulos 1 a 4 se despliega en el VPS con los datos de acceso y la base que provee la cátedra. El grupo construye la imagen en el servidor —o la construye en otro lado y la lleva— y la corre con `docker run`; el grupo elige y lo documenta. Queda accesible desde afuera en la URL que se declara en `DEPLOY.md`. Una vez arriba, la base se carga ejecutando dentro del contenedor el comando de datos iniciales que indica el README del TP (`docker exec libreria …`). El despliegue **tiene que seguir en línea hasta que se publiquen las notas**, porque la corrección lo prueba.

`DEPLOY.md` incluye: la URL pública, el día y la hora del despliegue, los comandos `docker build` y `docker run` completos —con el nombre del archivo de variables y sin su contenido—, cualquier cambio que se le haya hecho al TP para que corra en el contenedor, y cómo se actualiza la aplicación: se construye la imagen de nuevo, se elimina el contenedor y se lo vuelve a correr. Los datos no se pierden, porque viven en la base del servidor y no en el contenedor.

Endpoints de referencia

El grupo **abierto** no pide sesión (el catálogo es público, y `/auth` tiene que poder usarse para iniciarla). El **protegido** lleva el control del cambio obligatorio de contraseña montado una sola vez.

Grupo	Método	Ruta	Quién	Códigos
Abierto	GET	`/health/live`	Cualquiera	200
Abierto	GET	`/health/ready`	Cualquiera	200, 503
Abierto	POST	`/auth/registro`	Cualquiera	201, 409, 422
Abierto	POST	`/auth/login`	Cualquiera	200, 401, 422, 429
Abierto	POST	`/auth/logout`	Cualquiera	200
Abierto	GET	`/auth/yo`	Autenticado	200, 401
Abierto	POST	`/auth/cambiar-password`	Autenticado	200, 401, 422
Abierto	GET	`/libros` (`page`, `size`, `titulo`, `genero_id`)	Cualquiera	200, 422
Abierto	GET	`/libros/{id}`	Cualquiera	200, 404, 422

Grupo	Método	Ruta	Quién	Códigos
Abierto	GET	`/editoriales`, `/generos` (`page`, `size`)	Cualquiera	200, 422
Abierto	GET	`/editoriales/{id}/libros`, `/generos/{id}/libros`, `/autores/{id}/libros`	Cualquiera	200, 404
Protegido	POST	`/usuarios` (alta de personal, con contraseña provisoria)	ADMIN	201, 409, 422
Protegido	GET	`/usuarios/{id}/roles`	ADMIN	200, 404
Protegido	POST	`/usuarios/{id}/roles`	ADMIN	201, 404, 409, 422
Protegido	DELETE	`/usuarios/{id}/roles/{rol}`	ADMIN	200, 404, 409
Protegido	GET	`/eventos` (`page`, `size`)	ADMIN	200, 422
Protegido	POST	`/editoriales`, `/generos`, `/autores`	ADMIN	201, 409, 422
Protegido	PATCH	`/editoriales/{id}`, `/autores/{id}`	ADMIN	200, 404, 409, 422
Protegido	POST	`/libros` (con `autores` y `stock_inicial`)	ADMIN	201, 409, 422
Protegido	PATCH	`/libros/{id}` (título y precio; el stock no)	ADMIN	200, 404, 409, 422
Protegido	DELETE	`/libros/{id}`	ADMIN	204, 404
Protegido	POST	`/libros/{id}/reponer`	ADMIN	200, 404, 409, 422
Protegido	GET	`/libros/{id}/movimientos`	ADMIN	200, 404
Protegido	POST	`/clientes`	VENDEDOR, ADMIN	201, 409, 422
Protegido	GET, PATCH	`/clientes/{id}` (con su perfil)	VENDEDOR, ADMIN	200, 404, 409, 422
Protegido	POST	`/clientes/{id}/perfil`	VENDEDOR, ADMIN	201, 404, 409
Protegido	GET	`/clientes/{id}/ventas`	VENDEDOR, ADMIN	200, 404

Grupo	Método	Ruta	Quién	Códigos
Protegido	POST	`/ventas` (con `renglones`)	VENDEDOR, ADMIN	201, 409, 422
Protegido	GET	`/ventas` (`page`, `size`)	VENDEDOR, ADMIN	200, 422
Protegido	GET	`/ventas/{id}`	El personal, o el cliente dueño	200, 403, 404
Protegido	GET	`/mis-ventas` (`page`, `size`)	Autenticado	200, 422

Toda ruta del grupo protegido responde además 401 sin sesión y 403 con el cambio de contraseña pendiente.

> **EL DETALLE DE UNA VENTA NO LLEVA `REQUIERE_ROL`**
>
> Un cliente puede ver su propia venta y no tiene un rol de personal. Si la ruta exigiera VENDEDOR o ADMIN, el dueño recibiría «no tenés permiso» cuando sí lo tiene. La decisión la toma el servicio: primero quién es y qué relación tiene con la venta (403 si no es ni personal ni dueño), y recién después qué se devuelve.

Ejemplo del cuerpo de `POST /ventas`. Es el mismo del TP: viajan el id del cliente y los datos de cada renglón; no viajan el precio, el total ni la fecha.

```
{
  "cliente_id": 2,
  "renglones": [
    { "libro_id": 1, "cantidad": 2 },
    { "libro_id": 4, "cantidad": 1 }
  ]
}
```

Ejemplo del cuerpo de `POST /libros`. Viaja `stock_inicial` en lugar de `stock`, y el alta deja el movimiento «Stock inicial»:

```
{
  "isbn": "978-950-00-0006-6",
  "titulo": "Los que aman, odian",
  "precio": "21000.00",
  "stock_inicial": 4,
  "editorial_id": 1,
  "genero_id": 1,
  "autores": [
    { "autor_id": 3, "rol": "coautor", "orden": 1 },
    { "autor_id": 4, "rol": "coautor", "orden": 2 }
  ]
}
```

La respuesta del login no trae el token: viaja en el encabezado `Set-Cookie`.

```
{ "mensaje": "Sesión iniciada", "debe_cambiar_password": false, "expira_en_segundos":
1800 }
```

Un listado, `GET /libros?size=2`, con el envoltorio de la sección 2.9. Con `page=4` respondería 422: «La página 4 no existe: hay 3».

```
{
  "items": [
    { "id": 1, "isbn": "978-950-00-0001-1", "titulo": "Rayuela", "precio": "25000.00",
"stock": 10 },
    { "id": 2, "isbn": "978-950-00-0002-8", "titulo": "Antología de la literatura
fantástica", "precio": "30000.00", "stock": 5 }
  ],
  "total": 5,
  "page": 1,
  "size": 2,
  "pages": 3
}
```

Datos iniciales

`seed.py` vacía las tablas y carga, como mínimo, estos datos, que los casos de prueba usan. Las contraseñas son de mentira y viven sólo en ese archivo. Es parte de la puesta en marcha del proyecto y no un extra para probar.

Qué	Datos
Usuarios	`admin@example.com` y `admin2@example.com` (ADMIN), `vendedor@example.com` (VENDEDOR), `ana@example.com` (CLIENTE) y `nuevo@example.com` (VENDEDOR, con el cambio de contraseña obligatorio)

Qué	Datos
Editoriales, géneros, autores	Los del TP: 3 editoriales, 3 géneros y 6 autores
Libros	Los cinco del TP, con su stock actual: Rayuela 10, Antología de la literatura fantástica 5, Ficciones 8, Respiración artificial 3 y Bestiario 1
Clientes	Lucía Fernández (con perfil) y Tomás Ríos, sin usuario; Ana Paz, con el usuario `ana@example.com`
Venta	La 1, de Lucía, registrada por `vendedor@example.com`: 1 Rayuela y 2 Ficciones, total 69000.00
Movimientos	Un «Stock inicial» por libro. Rayuela y Ficciones se cargan con 11 y 10 ejemplares, y la venta 1 deja dos movimientos de salida (−1 y −2), de modo que `stock` coincide con la suma de movimientos en los cinco libros
Historial de seguridad	Una línea con el actor `SISTEMA`: «Carga de los datos iniciales»

Requisitos no funcionales

Requisito	Detalle
Asincronía	Todos los handlers con `async def`; bcrypt siempre fuera del bucle
Credenciales	Ninguna contraseña ni clave de firma en el código —tampoco como valor por defecto—, en `alembic.ini`, en el repositorio ni en la imagen
Token	Sólo en la cookie `HttpOnly`; nunca en el cuerpo ni leído de la cabecera `Authorization`
Esquema	Sólo con Alembic; la aplicación no usa `create_all`
Transacciones	Ningún repositorio confirma; ningún servicio construye su Unit of Work; el Unit of Work no cierra la sesión
Carga perezosa	Prohibida: toda relación que se lee se precarga en el repositorio
SQL	Nunca armado por concatenación con datos del usuario
Errores	Mensajes propios; nunca el texto de la base ni una traza; ninguna entrada inválida termina en un 500
Unicidad	Garantizada por la base; no se verifica con una consulta previa
Contenedor (despliegue del TP de los capítulos 1 a 4)	Imagen base `python:3.12-alpine`; la aplicación no corre como root; ningún secreto dentro de la imagen

Requisito	Detalle
Diagnóstico	Las consultas se pueden ver en la consola (`SQL_ECHO=true` o equivalente)

Casos de prueba mínimos

El proyecto incluye archivos `.http` en `test/` que demuestran los casos C, cada uno con la respuesta esperada en un comentario, y un `test/solo_agregado.sql` para el caso C-14. Antes de cada archivo se corre `python seed.py`. Los casos D se demuestran sobre **el TP de los capítulos 1 a 4 desplegado en el VPS**, con los comandos y las salidas guardados en la carpeta `evidencia/` del despliegue.

Caso	Qué se demuestra	Resultado esperado
C-01	Un parámetro de ruta que no es entero, con la sesión iniciada	422, sin ejecutar el handler
C-02	Registro; correo repetido de usuario y de cliente	201; 409 con mensaje propio, y el registro rechazado no deja ninguna fila suelta
C-03	La cookie del login	`Set-Cookie` con `HttpOnly`, `SameSite=Lax` y `Max-Age`; el cuerpo sin token; sin cookie o con `Authorization: Bearer`, 401
C-04	Correo existente con contraseña mala, y correo inexistente	401 con el mismo cuerpo y un tiempo parecido
C-05	Seis intentos fallidos seguidos	El sexto, 429 con `Retry-After` y en pocos milisegundos; los cinco fallos están en `intento_acceso`
C-06	401 contra 403, y lo propio	Sin sesión, 401; un vendedor que crea un libro, 403; la cliente ve su venta (200) y no la ajena (403)
C-07	Revocar un rol con la sesión abierta; vencimientos inválidos	La misma cookie, 403 en la petición siguiente; `/auth/yo` muestra la diferencia; la fila revocada sigue. Un vencimiento en el pasado o sin zona horaria, 422
C-08	Autoprotección	El administrador no se revoca su ADMIN: 409
C-09	Cambio obligatorio	Bloqueado (403) salvo `/auth`; después del cambio, `debe_cambiar_password` queda en falso y la misma cookie sirve para las rutas protegidas (200), sin iniciar sesión de nuevo

Caso	Qué se demuestra	Resultado esperado
C-10	Venta válida con dos renglones	201 con los precios copiados y el total; el stock de cada libro bajó, hay un movimiento por renglón y el aviso sale en el registro después de la respuesta
C-11	Venta con Rayuela (1 ejemplar, alcanza) y Bestiario (2 ejemplares, hay 1)	409; no hay venta nueva, ni descuentos, ni movimientos: el stock de Rayuela sigue en 10 y la suma de movimientos coincide con el stock en los cinco libros
C-12	Venta con un cliente o un libro inexistente, o con un libro dado de baja	409 en los tres, sin dejar escrituras
C-13	Reponer un libro; darlo de baja; volver a cargar el mismo ISBN	Reposición 200 con su movimiento; baja 204; el libro deja de aparecer en el catálogo; el alta con el mismo ISBN, 201; un segundo vigente con ese ISBN, 409; la venta vieja sigue mostrando el título
C-14	`UPDATE` y `DELETE` sobre `movimiento_stock` en `psql`	Los dos, rechazados por el disparador
C-15	Detalle de una venta con 2 y con 4 renglones	La misma cantidad de consultas en la consola
C-16	Búsqueda con `' OR '1'='1`; paginación con filtro; PATCH parcial; PATCH del stock	Lista vacía con `total` 0; con `titulo=ficc&size=1`, `total` 1 y `pages` 1, y `page=2` da 422; `size=500`, 422; sólo cambia lo enviado; el PATCH con `stock`, 422
D-01	`docker images` con la imagen base y con la del proyecto	`python:3.12-alpine` por debajo de los 100 MB, y el `Dockerfile` parte de ella
D-02	Servidor limpio: `docker build` y `docker run`; después `docker ps`	El contenedor en estado `healthy`; en `docker logs`, las migraciones del TP aplicadas al arrancar
D-03	Desde afuera del VPS, en la URL pública: `GET /health/ready`, `GET /libros/` con los datos iniciales cargados y `POST /ventas/` con el cuerpo de ejemplo del TP	200; 200 con la lista de libros; 201 con los precios copiados de cada libro y el total calculado
D-04	`docker rm -f libreria` y el mismo `docker run` otra vez	Vuelve a estar `healthy`, los datos siguen —la base no vive en el contenedor— y las migraciones no se repiten
D-05	Correr la imagen sin `DATABASE_URL`	El contenedor no queda sirviendo: termina con un error, porque el TP no trae ninguna dirección de base por defecto

Caso	Qué se demuestra	Resultado esperado
D-06	Contenido de la imagen	No hay `.env` (`docker run --rm --entrypoint ls <imagen> -a /app`) y `docker run --rm --entrypoint env <imagen>` no muestra `DATABASE_URL` ni ninguna contraseña
D-07	Usuario del proceso	`docker exec libreria id` no es root
D-08	La venta registrada en el caso D-03	El aviso del comprobante aparece en `docker logs libreria` después de la línea de la respuesta

VARIAS SESIONES A LA VEZ

Con la extensión «REST Client», cada login se nombra con `# @name` y los pedidos de esa sesión llevan `# @no-cookie-jar` y el encabezado `Cookie: {{loginAdmin.response.headers.Set-Cookie}}` . Así un mismo archivo puede tener la sesión del administrador y la del vendedor al mismo tiempo, que es lo que necesita el caso C-07.

Criterios de evaluación

Criterio	Peso	Qué se mira
Proyecto, configuración y ciclo de vida	6 %	Estructura, `Settings` sin secretos por defecto, motor y fábrica creados en el lifespan, `get_session` , salud
Modelos, relaciones y migraciones	10 %	Restricciones, índices únicos parciales, navegación, dos migraciones nuevas (una a mano), upgrade y downgrade
Contratos y CRUD	8 %	Contratos separados, `extra="forbid"` , códigos, ninguna entrada inválida en 500, PATCH parcial, envoltorio y paginación de la sección 2.9
Autenticación	14 %	bcrypt fuera del bucle y con semáforo, orden del login, límite, hash ficticio, cookie `HttpOnly`
Autorización	12 %	401 y 403, roles vigentes en la base, autoprotección, dos grupos de rutas, cambio obligatorio de contraseña, lo propio
Repositorios y Unit of Work	16 %	Dos jerarquías, clase del modelo, `flush` y no `commit` , precarga en el repositorio, quién crea, abre, confirma y cierra, operaciones de varias escrituras, un solo punto de escritura del stock
Integridad	10 %	Disparador de sólo agregado; intento de acceso en su propia sesión con su comentario; eventos en la misma transacción

Criterio	Peso	Qué se mira
Lecturas sin N+1 y asincronía	6 %	Consultas fijas en el detalle; ninguna llamada bloqueante; aviso en segundo plano visible
Contenedor: la imagen	11 %	Sobre el TP de los capítulos 1 a 4: imagen base `python:3.12-alpine`, capas en orden, usuario sin privilegios, sin secretos en la imagen, migraciones al arrancar, `HEALTHCHECK`, y ejecución con `docker run` y las variables por `--env-file`
Despliegue en el VPS	7 %	El TP de los capítulos 1 a 4 en línea en la URL declarada, `DEPLOY.md` completo, los casos D con su evidencia

Un proyecto que no levanta, que no crea su esquema con `alembic upgrade head` sobre una base vacía o que contiene credenciales reales en el repositorio, en la imagen o en el zip no se corrige hasta que se subsane. Si el despliegue no está accesible en el momento de la corrección, el criterio de despliegue en el VPS se evalúa sólo sobre lo que consta en `DEPLOY.md` y en la carpeta `evidencia/` del despliegue, y los casos D que no se puedan comprobar no suman.

Formato de entrega

La entrega es **una por grupo**, en **un único archivo** `.zip` que sube **un solo integrante** al campus. El archivo se llama `Parcial_Cap1a6_<NombreDelGrupo>.zip`; la fecha límite se publica en el campus.

El `.zip` contiene **dos carpetas**, porque son dos proyectos distintos:

- **`parcial/`**, el proyecto ampliado con todo lo que pide este parcial, salvo el despliegue: el código fuente completo, `requirements.txt`, `env.example`, `alembic.ini`, la carpeta `migrations/` y la carpeta `test/`. Incluye un `README.md` con los nombres y apellidos de los integrantes y los pasos para levantarlo en la máquina del grupo desde cero (entorno virtual, dependencias, `.env` con cómo generar la clave de firma, `alembic upgrade head`, `python seed.py` y arranque del servidor), y una carpeta `evidencia/` con la captura o el fragmento de consola del caso C-15 (las consultas del detalle de una venta con 2 renglones y de otra con 4) y la captura de `psql` del caso C-14.
- **`despliegue/`**, el **TP de los capítulos 1 a 4** tal como se entregó, con el `Dockerfile`, el `.dockerignore`, un `env.example` con las variables del contenedor (la forma de `DATABASE_URL`, sin valores reales) y el `DEPLOY.md` que pide la consigna de despliegue. Incluye una carpeta `evidencia/` con la salida de cada caso D.

El `.zip` **no contiene** las carpetas `.venv` ni `__pycache__` —en ninguna de las dos carpetas del proyecto— ni el archivo `.env`: el entorno virtual se reconstruye con `requirements.txt`, y los `__pycache__` los genera Python solo. Si incluye la carpeta `.git`, su historial también se revisa: una contraseña que alguna vez se subió y se borró después sigue estando en el historial, y en ese caso se pide además cambiarla.